

```
hbase(main):019:0# scan 'Social'
ROW                                COLUMN+CELL
User1                             column=Connection:Friend, timestamp=1273818803385, value=User2
User1                             column=Connection:Relative, timestamp=1273818809981, value=User3
User1                             column=Details:City, timestamp=1273818796485, value=Bangalore
User1                             column=Details:Name, timestamp=1273818788378, value=John
User2                             column=Details:Birthplace, timestamp=1273818824461, value=Delhi
User2                             column=Details:Photograph, timestamp=1273818832131, value=/images/image002.jpeg
User2                             column=Notification:Birthday, timestamp=1273818817187, value=User1
2 row(s) in 0.0580 seconds
hbase(main):020:0#
```

Figure 2: Scan output of the "Social" table

```
Birthday','User1'
put 'Social','User2','Details:Birthplace','Delhi'
put 'Social','User2','Details:Photograph','/images/image002.jpeg'
```

This set of queries, when run successfully, will add two records to the database—one each for *User1* and *User2*. Each of these records will contain data about the user, which can be in one of the defined families (*Details*, *Connection* and *Notification*).

Each row value has a qualifier (*Name*, *City*, *Relative*, etc), and a value associated with it, thereby creating data triplets *<rowname, qualifier, value>*. To display the data in the table via the CLI, use the *scan* and *get* queries, which are similar to the SQL *SELECT* statement:

```
scan 'Social'
```

This will display the complete table, as shown in Figure 2.

```
get 'Social','User1'
```

This query will display all the data in the record named 'User1'. Each record that has been inserted into the table also has a time-stamp associated with it, which helps in maintaining atomicity, and is also for indexing while sorting and searching.

 **Note:** Type *help* at the HBase CLI prompt for a complete list of commands, and their usage. You can also find a complete list of CLI commands at <http://wiki.apache.org/hadoop/Hbase/Shell>.

Using HBase in a Java application

The following example provides an overview of using HBase in a Java application. Open your favourite IDE (preferably NetBeans or Eclipse), and create a new Java project. Add the following libraries to the project (from the HBase *lib* folder).

- commons-logging-1.0.4.jar
- hadoop-0.20.1-core.jar
- hbase-0.20.1.jar
- log4j-1.2.13.jar

Import the sub-packages under the *org.apache.hadoop.hbase* package, so that we can use the required functions.

Accompanying this article is a file containing a class named *HBase_example*. Find the file on the LFY website at http://www.linuxforu.com/article_source_code/june10/

ride_on_hadoop/HBase_example. This class has an empty constructor and three functions for inserting, searching and selecting data. A main function calls these functions with sample data. In each of the functions, we make a connection to the database using the following code:

```
HBaseConfiguration hbaseConfig = new HBaseConfiguration();
HTable htable = new HTable(hbaseConfig, "Social");
```



Note: The code assumes that the *Social* table already exists, created by the CLI commands given above.

Inserting data

The *insert()* function is similar to an *INSERT* query in SQL, and does the same work as the CLI query. The arguments passed to this function are: *resource*, *family*, *relationship*, *value*.

```
htable.setWriteBufferSize(1024 * 1024 * 12);
byte [] rowkey = Bytes.toBytes(resource);
Put put = new Put(rowkey);
put.add(Bytes.toBytes(family), Bytes.toBytes(relationship), Bytes.toBytes(value));
htable.put(put);
```

First, we convert the *resource* variable into a stream of bytes, which can be thought of as the primary key for that entry in the table. Then we create an object of the *Put* class, using the bytes for the *resource* (in the variable *rowkey*). We then use the *add* function of the *Put* class to add the triplet to the variable, and add it into the database using the *htable.put()* function.

Selecting and displaying data

The *select()* function is similar to an SQL *SELECT * FROM tablename* query, and is used to display the contents of the table as they are stored in the table.

```
Scan s = new Scan();
s.addColumn(Bytes.toBytes(family));
ResultScanner scanner = htable.getScanner(s);
for (Result rr : scanner) {
    rr.value();
    System.out.println(rr);
}
```